

Artificial Intelligence and Data Engineering for Orbital and Space Observation Systems

Laboratory 9: Transfer Learning for Satellite Image Classification

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In the previous laboratory, you built your first deep learning pipeline for satellite image classification.

You trained a small convolutional neural network from scratch.

This means that the model started with random weights and had to learn visual patterns only from the small dataset prepared during the laboratory.

This approach is useful for learning, but in real Earth Observation systems it has limitations:

- training from scratch may require many images,
- training may take a long time,
- small datasets may lead to overfitting,
- model quality may be limited by the available training data.

In this laboratory, you will use a different approach:

```
transfer learning
```

Transfer learning means using a neural network that was already trained on a large image dataset and adapting it to a new task.

Instead of training all layers from zero, we reuse visual knowledge already learned by a large model.

In this laboratory, you will use:

```
ResNet18
```

ResNet18 is a convolutional neural network architecture commonly used for image classification. You will use a pretrained version of ResNet18 and adapt it to classify three satellite image classes:

```
forest  
residential  
river
```

The main engineering question is:

Is it better to train a small CNN from scratch, or to adapt a pretrained model using transfer learning?

You will build the following pipeline:

```
prepare dataset -> load pretrained ResNet18 -> replace classifier  
-> train classifier head -> evaluate -> fine-tune -> compare with SimpleCNN
```

2 Dataset

This laboratory uses the same processed EuroSAT subset as Laboratory 8.

The expected dataset structure is:

```
data/processed/images/  
  train/  
    forest/  
    residential/  
    river/  
  test/  
    forest/  
    residential/  
    river/
```

If you already completed Laboratory 8, you may reuse the same dataset.

If not, first run the dataset preparation scripts from Laboratory 8:

```
python3 src/data/download_eurosat.py  
python3 src/data/prepare_image_dataset.py
```

The dataset should contain:

- 240 training images,
- 60 testing images,
- 3 classes.

3 Environment Setup

Install required libraries:

```
python3 -m pip install torch torchvision pillow numpy matplotlib scikit-learn
```

All commands should be executed from the root directory of your project.

If you use Windows and `python3` does not work, use:

```
python
```

instead.

4 Project Structure

The target project structure is:

```
data/
```

```
raw/
processed/
inference_samples/

models/
reports/

src/
  data/
    download_eurosat.py
    prepare_image_dataset.py

  vision/
    image_dataset.py
    cnn_model.py
    train_cnn.py
    predict_cnn.py
    train_transfer.py
    create_transfer_gallery.py
    train_finetune.py
    create_gradcam.py
    predict_transfer.py
```

Create missing folders if necessary:

```
mkdir -p models reports
mkdir -p src/vision
```

5 Task 1: Verify Dataset Availability

Before training a pretrained model, verify that the dataset exists.

Run:

```
ls data/processed/images/train
ls data/processed/images/test
```

You should see:

```
forest
residential
river
```

If the folders do not exist, return to Laboratory 8 and prepare the dataset first.

Task

Verify that:

- the training folder exists,
- the testing folder exists,
- all three classes are available,
- each class contains image files.

6 Task 2: Reuse the PyTorch Dataset

This laboratory reuses the PyTorch Dataset created in Laboratory 8:

```
src/vision/image_dataset.py
```

The dataset should define:

```
class EuroSATDataset(Dataset):  
    ...
```

This class should:

- read image paths from folders,
- map class names to integer labels,
- load images,
- apply transforms,
- return image tensor and label.

Task

Run dataset inspection again:

```
python3 src/vision/image_dataset.py
```

Expected output:

```
=== PyTorch Dataset Inspection ===  
Classes: ['forest', 'residential', 'river']  
Class mapping: {'forest': 0, 'residential': 1, 'river': 2}  
Number of samples: 240  
First image tensor shape: torch.Size([3, 64, 64])  
First label: 0
```

7 Task 3: Why Transfer Learning?

In Laboratory 8, your SimpleCNN learned all filters from scratch.

This means that the network had to learn basic visual concepts such as:

- edges,
- corners,
- textures,
- shapes,
- local patterns.

from only a small dataset.

A pretrained CNN already learned many of these patterns from a very large dataset.

For example, early layers of a pretrained model often detect:

- edges,
- color transitions,
- repeated textures,
- simple object boundaries.

These patterns are useful not only for ordinary photographs, but also for satellite imagery. For example:

- forest texture may appear as repeated green patterns,
- rivers may appear as elongated curved structures,
- residential areas may contain regular geometric patterns.

Transfer learning uses this idea:

Reuse general visual knowledge and adapt only the final classifier to the new task.

8 Task 4: Create Transfer Learning Training Script

Create:

```
src/vision/train_transfer.py
```

Add imports:

```
from pathlib import Path
import time

import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import transforms, models

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

from src.vision.image_dataset import EuroSATDataset
```

Add configuration:

```
TRAIN_DIR = Path("data/processed/images/train")
TEST_DIR = Path("data/processed/images/test")

BATCH_SIZE = 16
EPOCHS = 8
LEARNING_RATE = 0.001

MODEL_PATH = Path("models/resnet18_transfer.pt")
CLASS_NAMES_PATH = Path("models/resnet18_classes.txt")
REPORT_PATH = Path("reports/transfer_learning_report.txt")
CONFUSION_MATRIX_PATH = Path("reports/transfer_confusion_matrix.png")
```

9 Task 5: Create DataLoaders for ResNet18

ResNet18 expects images of size:

224 x 224

In Laboratory 8, your SimpleCNN used:

64 x 64

Therefore, we must resize images before giving them to ResNet18.

Note that the processed dataset from Laboratory 8 stores images as 64x64 pixels. In this laboratory, these images are resized to 224x224 pixels before being passed to ResNet18. This is acceptable here because we want to compare all models using the same prepared dataset.

Add:

```
def create_dataloaders():
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor()
    ])

    train_dataset = EuroSATDataset(
        root_dir=TRAIN_DIR,
        transform=transform
    )

    test_dataset = EuroSATDataset(
        root_dir=TEST_DIR,
        transform=transform
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    print("=== Transfer Learning DataLoader ===")
    print(f"Training samples: {len(train_dataset)}")
    print(f"Testing samples: {len(test_dataset)}")
    print(f"Classes: {train_dataset.class_names}")
    print(f"Class mapping: {train_dataset.class_to_index}")

    images, labels = next(iter(train_loader))

    print(f"Batch image shape: {images.shape}")
    print(f"Batch label shape: {labels.shape}")

    return train_loader, test_loader, train_dataset.class_names
```

What Happens Here?

The batch shape should be:

```
torch.Size([16, 3, 224, 224])
```

This means:

- 16 images in one batch,
- 3 RGB channels,
- each image resized to 224x224 pixels.

This input format matches the expected input size of ResNet18.

10 Task 6: Select Device

Add:

```
def get_device():
    if torch.cuda.is_available():
        return torch.device("cuda")

    return torch.device("cpu")
```

This allows PyTorch to use GPU if available.

If no GPU is available, training will run on CPU.

11 Task 7: Load Pretrained ResNet18

Add:

```
def create_transfer_model(num_classes):
    model = models.resnet18(
        weights=models.ResNet18_Weights.DEFAULT
    )

    return model
```

What Happens Here?

This downloads and loads a pretrained ResNet18 model.

The model was trained on a large image dataset.

It already contains learned filters useful for recognizing visual patterns.

However, the original ResNet18 classifier predicts:

```
1000 ImageNet classes
```

Our task has only:

```
3 classes
```

Therefore, we must modify the final classifier layer.

Important Note

At this point, the model is loaded but not yet adapted to our three-class problem.

The original ResNet18 classifier still expects to predict:

```
1000 ImageNet classes
```

The next tasks will freeze the pretrained feature extractor and replace the final classifier with a new layer for:

```
forest  
residential  
river
```

12 Task 8: Freeze Feature Extractor

In transfer learning, we often freeze most of the pretrained model.

This means:

The pretrained layers keep their learned weights. Only the final classifier is trained.

Replace the previous version of:

```
create_transfer_model()
```

with:

```
def create_transfer_model(num_classes):  
    model = models.resnet18(  
        weights=models.ResNet18_Weights.DEFAULT  
    )  
  
    for parameter in model.parameters():  
        parameter.requires_grad = False  
  
    return model
```

Why Freeze Layers?

Freezing layers is useful because:

- training is faster,
- fewer parameters are updated,
- the model keeps general visual knowledge,
- overfitting risk is reduced for small datasets.

In this stage, ResNet18 acts as a pretrained feature extractor.

13 Task 9: Replace the Classifier Head

The final layer of ResNet18 must be replaced.

The original model outputs 1000 classes.

We need 3 output classes.

Replace the previous version of:


```
create_transfer_model()
```

with:

```
def create_transfer_model(num_classes):  
    model = models.resnet18(  
        weights=models.ResNet18_Weights.DEFAULT  
    )  
  
    for parameter in model.parameters():  
        parameter.requires_grad = False  
  
    input_features = model.fc.in_features  
  
    model.fc = nn.Linear(  
        input_features,  
        num_classes  
    )  
  
    return model
```

What Happens Here?

The line:

```
input_features = model.fc.in_features
```

checks how many inputs the original classifier receives.

For ResNet18, this value is usually:

```
512
```

The new classifier:

```
nn.Linear(input_features, num_classes)
```

maps pretrained visual features into our three classes:

```
forest  
residential  
river
```

14 Task 9.1: Inspect Trainable Parameters

Before training, verify which parts of the model will actually be updated.

Add:

```
def print_trainable_parameters(model):  
    print("=== Trainable Parameters ===")  
  
    trainable_count = 0  
    total_count = 0  
  
    for name, parameter in model.named_parameters():
```

```

        total_count += parameter.numel()

        if parameter.requires_grad:
            trainable_count += parameter.numel()
            print(name)

    print(f"Trainable parameters: {trainable_count}")
    print(f"Total parameters: {total_count}")

```

Then call:

```
print_trainable_parameters(model)
```

after creating the model in:

```
main()
```

What Happens Here?

This diagnostic shows which model parameters will be updated during training.

In the frozen transfer learning setup, you should mainly see:

```

fc.weight
fc.bias

```

This confirms that:

- pretrained convolutional layers remain frozen,
- only the new classifier head is being trained.

Later, during fine-tuning, you should also see parameters from:

```
layer4
```

This confirms that deeper visual features are also being updated.

15 Task 10: Train the Classifier Head

Add the training function:

```

def train_model(model, train_loader, device):
    loss_function = nn.CrossEntropyLoss()

    optimizer = torch.optim.Adam(
        model.fc.parameters(),
        lr=LEARNING_RATE
    )

    model.train()

    start_time = time.time()

    for epoch in range(EPOCHS):
        total_loss = 0.0

```

```

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = loss_function(outputs, labels)

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    average_loss = total_loss / len(train_loader)

    print(f"Epoch {epoch + 1}/{EPOCHS}, Loss: {average_loss:.4f}")

end_time = time.time()
training_time = end_time - start_time

print(f"Training time: {training_time:.2f} seconds")

return training_time

```

What Is Being Trained?

Because most layers are frozen, only the final classifier is trained.

This means:

- convolutional layers keep pretrained filters,
- the final layer learns how to classify EuroSAT classes,
- training should be faster than full model training.

16 Task 11: Evaluate the Model

Add:

```

def evaluate_model(model, test_loader, device, class_names):
    model.eval()

    all_labels = []
    all_predictions = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)

            outputs = model(images)

            _, predicted = torch.max(
                outputs,
                dim=1
            )

```

```

        all_labels.extend(labels.numpy())
        all_predictions.extend(predicted.cpu().numpy())

    accuracy = accuracy_score(
        all_labels,
        all_predictions
    )

    print("=== Transfer Learning Evaluation ===")
    print(f"Test samples: {len(all_labels)}")
    print(f"Accuracy: {accuracy:.4f}")

    matrix = confusion_matrix(
        all_labels,
        all_predictions
    )

    display = ConfusionMatrixDisplay(
        confusion_matrix=matrix,
        display_labels=class_names
    )

    CONFUSION_MATRIX_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    display.plot()
    plt.title("Transfer Learning Confusion Matrix")
    plt.tight_layout()
    plt.savefig(CONFUSION_MATRIX_PATH)
    plt.close()

    print(f"Saved confusion matrix: {CONFUSION_MATRIX_PATH}")

    return accuracy, all_labels, all_predictions

```

Why Use a Confusion Matrix?

Accuracy tells us one number.

The confusion matrix shows:

- which classes are classified correctly,
- which classes are confused,
- whether errors are systematic,
- whether one class is more difficult than others.

This is important in satellite imagery because some visual classes may overlap.

For example:

- rivers may be confused with dark forest regions,
- residential areas may be confused with roads or industrial patterns,
- forest texture may vary between regions.

17 Task 12: Save Model and Class Names

Add:

```
def save_model(model, class_names):
    MODEL_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    torch.save(
        model.state_dict(),
        MODEL_PATH
    )

    with open(CLASS_NAMES_PATH, "w") as f:
        for class_name in class_names:
            f.write(class_name + "\n")

    print("=== Saving Transfer Model ===")
    print(f"Saved model: {MODEL_PATH}")
    print(f"Saved classes: {CLASS_NAMES_PATH}")
```

18 Task 13: Save Training Report

Add:

```
def save_report(accuracy, training_time, class_names):
    REPORT_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    with open(REPORT_PATH, "w") as f:
        f.write("TRANSFER LEARNING REPORT\n")
        f.write("=====\n\n")

        f.write("Model: ResNet18 pretrained on ImageNet\n")
        f.write("Training mode: frozen feature extractor\n")
        f.write("Classifier: replaced final linear layer\n\n")

        f.write(f"Classes: {class_names}\n")
        f.write(f"Epochs: {EPOCHS}\n")
        f.write(f"Learning rate: {LEARNING_RATE}\n")
        f.write(f"Training time: {training_time:.2f} seconds\n")
        f.write(f"Test accuracy: {accuracy:.4f}\n\n")

        f.write("Interpretation:\n")
        f.write(
            "The model reused pretrained visual features and trained only "
            "a new classifier head for the EuroSAT subset.\n"
        )

    print(f"Saved report: {REPORT_PATH}")
```

19 Task 14: Add Main Function

Add:

```
def main():
    train_loader, test_loader, class_names = create_dataloaders()

    device = get_device()

    print(f"Using device: {device}")

    model = create_transfer_model(
        num_classes=len(class_names)
    )

    print_trainable_parameters(model)

    model = model.to(device)
    training_time = train_model(
        model,
        train_loader,
        device
    )

    accuracy, all_labels, all_predictions = evaluate_model(
        model,
        test_loader,
        device,
        class_names
    )

    save_model(
        model,
        class_names
    )

    save_report(
        accuracy,
        training_time,
        class_names
    )

if __name__ == "__main__":
    main()
```

Run:

```
python3 -m src.vision.train_transfer
```

Expected Output

You should see something similar to:

```
=== Transfer Learning DataLoader ===
Training samples: 240
Testing samples: 60
Classes: ['forest', 'residential', 'river']
```

```
Batch image shape: torch.Size([16, 3, 224, 224])
Using device: cpu
Epoch 1/8, Loss: 0.9321
Epoch 2/8, Loss: 0.6124
Epoch 3/8, Loss: 0.4218
...
Training time: 45.37 seconds
=== Transfer Learning Evaluation ===
Test samples: 60
Accuracy: 0.9000
Saved confusion matrix: reports/transfer_confusion_matrix.png
Saved model: models/resnet18_transfer.pt
Saved classes: models/resnet18_classes.txt
Saved report: reports/transfer_learning_report.txt
```

The exact accuracy and training time may differ.

20 Task 15: Predict a Single Image Using Transfer Learning Model

Create:

```
src/vision/predict_transfer.py
```

Add imports:

```
from pathlib import Path

import torch
from torch import nn
from torchvision import transforms, models
from PIL import Image
import matplotlib.pyplot as plt
```

Add configuration:

```
MODEL_PATH = Path("models/resnet18_transfer.pt")
CLASS_NAMES_PATH = Path("models/resnet18_classes.txt")
```

Add class loading:

```
def load_class_names():
    if not CLASS_NAMES_PATH.exists():
        print(f"Error: class file not found: {CLASS_NAMES_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    with open(CLASS_NAMES_PATH, "r") as f:
        class_names = [
            line.strip()
            for line in f.readlines()
            if line.strip()
        ]

    return class_names
```

Add model loading:

```
def load_model(class_names):
    if not MODEL_PATH.exists():
        print(f"Error: model file not found: {MODEL_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    model = models.resnet18(weights=None)

    input_features = model.fc.in_features

    model.fc = nn.Linear(
        input_features,
        len(class_names)
    )

    state_dict = torch.load(
        MODEL_PATH,
        map_location="cpu"
    )

    model.load_state_dict(state_dict)

    model.eval()

    return model
```

Add prediction function:

```
def predict_image(model, class_names, image_path):
    path = Path(image_path)

    if not path.exists():
        print(f"Error: image not found: {path}")
        return

    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor()
    ])

    with Image.open(path) as image:
        image = image.convert("RGB")
        image_for_plot = image.copy()
        image_tensor = transform(image)

    image_tensor = image_tensor.unsqueeze(0)

    with torch.no_grad():
        outputs = model(image_tensor)

        probabilities = torch.softmax(
            outputs,
            dim=1
        )

    confidence, predicted_index = torch.max(
```



```

        probabilities,
        dim=1
    )

    predicted_class = class_names[
        predicted_index.item()
    ]

    print("=== Transfer Learning Prediction ===")
    print(f"Image: {image_path}")
    print(f"Predicted class: {predicted_class}")
    print(f"Confidence: {confidence.item():.4f}")

    plt.imshow(image_for_plot)
    plt.title(
        f"Transfer model prediction: {predicted_class}\n"
        f"Confidence: {confidence.item():.4f}"
    )
    plt.axis("off")
    plt.show()

```

Add main:

```

def main():
    class_names = load_class_names()

    model = load_model(class_names)

    image_path = "data/processed/images/test/forest/forest_0000.jpg"

    predict_image(
        model,
        class_names,
        image_path
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.predict_transfer
```

Expected Output

You should see:

```

=== Transfer Learning Prediction ===
Image: data/processed/images/test/forest/forest_0000.jpg
Predicted class: forest
Confidence: 0.9123

```

21 Task 16: Fine-Tune the Last ResNet Block

So far, you trained only the final classifier head.

Now you will create a second version of the training script. This version will allow the last ResNet block to adapt to satellite imagery.

This is called:

```
fine-tuning
```

Fine-tuning means that selected pretrained layers are unfrozen and trained further.

Task 16.1: Copy the Transfer Learning Script

Do not overwrite the previous transfer learning experiment.

First, create a copy of the script:

```
cp src/vision/train_transfer.py src/vision/train_finetune.py
```

On Windows PowerShell, you may use:

```
Copy-Item src/vision/train_transfer.py src/vision/train_finetune.py
```

This allows you to keep two separate experiments:

- `train_transfer.py` — frozen feature extractor,
- `train_finetune.py` — fine-tuned last ResNet block.

Task 16.2: Change Output Paths

In:

```
src/vision/train_finetune.py
```

modify the output paths:

```
MODEL_PATH = Path("models/resnet18_finetuned.pt")
CLASS_NAMES_PATH = Path("models/resnet18_finetuned_classes.txt")
REPORT_PATH = Path("reports/fine_tuning_report.txt")
CONFUSION_MATRIX_PATH = Path("reports/fine_tuning_confusion_matrix.png")
```

Also update the report description in `save_report()`:

```
f.write("Model: ResNet18 pretrained on ImageNet\n")
f.write("Training mode: fine-tuned last ResNet block\n")
f.write("Trainable layers: layer4 and final classifier\n\n")
```

Task 16.3: Replace the Existing Model Definition

Locate the latest version of:

```
create_transfer_model()
```

This function should currently:

- load pretrained ResNet18,
- freeze all pretrained parameters,
- replace the final classifier head.

Replace the entire function with:

```
def create_transfer_model(num_classes):
    model = models.resnet18(
        weights=models.ResNet18_Weights.DEFAULT
    )

    for parameter in model.parameters():
        parameter.requires_grad = False

    for parameter in model.layer4.parameters():
        parameter.requires_grad = True

    input_features = model.fc.in_features

    model.fc = nn.Linear(
        input_features,
        num_classes
    )

    return model
```

Task 16.4: Modify the Optimizer

In the fine-tuning version, the optimizer should update only trainable parameters.

In the function:

```
train_model()
```

replace the optimizer definition with:

```
optimizer = torch.optim.Adam(
    filter(
        lambda parameter: parameter.requires_grad,
        model.parameters()
    ),
    lr=LEARNING_RATE
)
```

Task 16.5: Run Fine-Tuning

Run:

```
python3 -m src.vision.train_finetune
```

Now you should have two separate models:

```
models/resnet18_transfer.pt
models/resnet18_finetuned.pt
```

and two separate confusion matrices:

```
reports/transfer_confusion_matrix.png
reports/fine_tuning_confusion_matrix.png
```

What Happens Here?

In this version, the model trains:

- the new final classifier,
- the last ResNet feature block: `layer4`.

All earlier ResNet layers remain frozen.

This means that the model still reuses general visual features learned from a large image dataset, but the deepest feature block is allowed to adapt to satellite imagery.

This is useful because higher-level features may need to change for Earth Observation scenes.

For example, the model may need to better represent:

- land-cover textures,
- river shapes,
- forest patterns,
- regular structures in residential areas.

Compared to the frozen feature extractor, fine-tuning usually:

- gives the model more flexibility,
- may improve accuracy,
- increases training time,
- increases the risk of overfitting on small datasets.

After training, you should have results from two separate experiments:

- frozen ResNet18 feature extractor from `train_transfer.py`,
- fine-tuned ResNet18 from `train_finetune.py`.

Compare:

- frozen feature extractor accuracy,
- fine-tuned model accuracy,
- frozen feature extractor training time,
- fine-tuned model training time,
- frozen feature extractor confusion matrix,
- fine-tuned model confusion matrix.

Analysis Questions

Answer:

- Did fine-tuning improve accuracy?
- Did training become slower?
- Did the confusion matrix change?
- Did any class improve more than others?
- Did any class become worse?
- Is fine-tuning worth the additional computational cost?
- Would you use the frozen model or the fine-tuned model in a small-data satellite classification problem?

Save your conclusions in:

```
reports/fine_tuning_analysis.txt
```

22 Task 17: Compare SimpleCNN and ResNet18 Transfer Learning

Now compare three approaches:

- SimpleCNN trained from scratch,
- ResNet18 with frozen feature extractor,
- ResNet18 with fine-tuning.

Create:

```
reports/model_comparison_lab9.txt
```

Use the following format:

```
MODEL COMPARISON
=====

1. SimpleCNN from scratch
Accuracy: ...
Training time: ...
Comments: ...

2. ResNet18 frozen feature extractor
Accuracy: ...
Training time: ...
Comments: ...

3. ResNet18 fine-tuned last block
Accuracy: ...
Training time: ...
Comments: ...
```

```
BEST ACCURACY:
```

```
...
```

```
FASTEST TRAINING:
```

```
...
```

```
BEST ENGINEERING TRADE-OFF:
```

```
...
```

Engineering Questions

Answer:

- Which model achieved the best accuracy?
- Which model trained fastest?
- Which model required the least implementation effort?
- Which model is most suitable for a small dataset?
- Which model is most suitable for a real operational system?
- When would you train a CNN from scratch?
- When would you use transfer learning?
- When would you fine-tune a pretrained model?

23 Closed-Set Classification Limitation

So far, all models in this laboratory were trained only on:

```
forest  
residential  
river
```

This creates a:

```
closed-set classification problem
```

This means that during inference, the model assumes that every input image belongs to one of the known classes.

Even if the input is:

- a completely different land-cover class,
- an unrelated image,
- sensor noise,
- corrupted input data,
- an out-of-distribution sample,

the model will still produce one of the known class predictions.

Why Is This Important?

Transfer learning improves feature extraction, but it does not automatically solve reliability or safety problems.

A pretrained model may still produce:

- incorrect predictions,
- overconfident predictions,
- unstable behavior on unknown inputs.

In operational Earth Observation systems, this may create engineering risks.

For example:

- false land-cover interpretation,
- missed anomalies,
- incorrect mission decisions,
- unreliable autonomous behavior.

24 Task 18: Test Unknown Inputs Again

Repeat the unknown input experiments from Laboratory 8.

Test:

```
data/raw/eurosat/2750/Highway/Highway_1.jpg
```

and:

```
data/inference_samples/noise.jpg
```

using:

```
python3 -m src.vision.predict_transfer
```

If the noise image does not exist, create it using the noise image script from Laboratory 8.

Task

Compare the behavior of:

- SimpleCNN,
- ResNet18 transfer learning model.

Answer:

- Does the transfer model also force a known class prediction?
- Does it give high confidence on unknown input?
- Is transfer learning enough to solve out-of-distribution detection?
- What additional mechanism would be needed in an operational system?

Save your answers in:

```
reports/unknown_input_transfer_analysis.txt
```

25 Add ImageNet Normalization

In previous experiments, images were converted into tensors using:

```
transforms.ToTensor()
```

This converts pixel values from:

```
0 ... 255
```

into:

```
0.0 ... 1.0
```

This is often sufficient when training a model from scratch.

However, in this laboratory we are not training a CNN from scratch.

We are using:

```
ResNet18 pretrained on ImageNet
```

This model was originally trained on millions of images using a specific input preprocessing pipeline.

One important part of that preprocessing is:

```
input normalization
```

What Is Normalization?

Normalization transforms pixel values so that each color channel has approximately the same statistical distribution as the data used during the original ImageNet training.

For RGB images, this means:

- red channel values are centered around a known mean,
- green channel values are centered around a known mean,
- blue channel values are centered around a known mean.

The model expects inputs that roughly match this distribution.

What Happens Without Normalization?

If normalization is not used:

- the pretrained filters may receive data in a different scale,
- feature activations may become less stable,
- the model may not fully reuse pretrained visual knowledge,
- accuracy may decrease,

- training may become less consistent.

In other words:

The model may still work, but it may not operate in the input regime it was originally trained for.

What Happens With Normalization?

If normalization is applied:

- input statistics become closer to ImageNet training data,
- pretrained filters may respond more naturally,
- feature extraction may become more reliable,
- training may become more stable,
- accuracy may improve.

This is especially important in transfer learning, because we want to reuse previously learned visual representations as effectively as possible.

Engineering Intuition

You can think of normalization as:

Calibrating the input sensor data so that the model sees data in the format it expects.

In real space systems, this is similar to sensor calibration:

- if the sensor is not calibrated, measurements may still exist, but interpretation may be less reliable;
- if the sensor is calibrated, measurements become more consistent and easier to interpret.

Task

Now modify your transform:

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Important: if you train the model with normalization, you must use the same normalization during prediction.

Therefore, update the transform in:

```
src/vision/predict_transfer.py
```

in the same way.

Retrain and compare results.

Answer:

- Did normalization improve accuracy?
- Did training become more stable?
- Why may pretrained models require specific preprocessing?

26 Train with More Data

Increase:

```
IMAGES_PER_CLASS = 100
```

in the dataset preparation script to:

```
IMAGES_PER_CLASS = 300
```

Then prepare the dataset again and retrain:

- SimpleCNN,
- ResNet18 transfer model.

Answer:

- Which model benefits more from additional data?
- Does the accuracy gap change?
- Does training time increase significantly?

27 Create a Prediction Gallery

A single prediction is useful, but it does not show how the model behaves across multiple classes.

Now create a visual gallery of predictions.

The gallery should show:

- several test images,
- the true class,
- the predicted class,
- the prediction confidence.

This helps visually inspect whether the model makes reasonable decisions.

Task 1: Create the Gallery Script

Create:

```
src/vision/create_transfer_gallery.py
```

Add imports:

```
from pathlib import Path

import torch
from torch import nn
from torchvision import transforms, models
from PIL import Image
import matplotlib.pyplot as plt
```

Add configuration:

```
TEST_DIR = Path("data/processed/images/test")

MODEL_PATH = Path("models/resnet18_transfer.pt")
CLASS_NAMES_PATH = Path("models/resnet18_classes.txt")

OUTPUT_PATH = Path("reports/transfer_prediction_gallery.png")
IMAGES_PER_CLASS = 3
```

Task 2: Load Class Names

Add:

```
def load_class_names():
    if not CLASS_NAMES_PATH.exists():
        print(f"Error: class file not found: {CLASS_NAMES_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    with open(CLASS_NAMES_PATH, "r") as f:
        class_names = [
            line.strip()
            for line in f.readlines()
            if line.strip()
        ]

    return class_names
```

Task 3: Load the Transfer Model

Add:

```
def load_model(class_names):
    if not MODEL_PATH.exists():
        print(f"Error: model file not found: {MODEL_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    model = models.resnet18(weights=None)
```

```

input_features = model.fc.in_features

model.fc = nn.Linear(
    input_features,
    len(class_names)
)

state_dict = torch.load(
    MODEL_PATH,
    map_location="cpu"
)

model.load_state_dict(state_dict)

model.eval()

return model

```

Task 4: Select Test Images

Add:

```

def select_test_images(class_names):
    selected_images = []

    for class_name in class_names:
        class_dir = TEST_DIR / class_name

        image_paths = sorted([
            path for path in class_dir.iterdir()
            if path.suffix.lower() in [".jpg", ".jpeg", ".png"]
        ])

        for image_path in image_paths[:IMAGES_PER_CLASS]:
            selected_images.append(
                (image_path, class_name)
            )

    return selected_images

```

Task 5: Predict One Image

Add:

```

def predict_image(model, class_names, image_path, transform):
    with Image.open(image_path) as image:
        image = image.convert("RGB")
        image_tensor = transform(image)

    image_tensor = image_tensor.unsqueeze(0)

    with torch.no_grad():
        outputs = model(image_tensor)

    probabilities = torch.softmax(
        outputs,
        dim=1
    )

```

```

    )

    confidence, predicted_index = torch.max(
        probabilities,
        dim=1
    )

    predicted_class = class_names[
        predicted_index.item()
    ]

    return predicted_class, confidence.item()

```

Task 6: Create and Save the Gallery

Add:

```

def create_gallery(model, class_names):
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])

    selected_images = select_test_images(class_names)

    rows = len(class_names)
    columns = IMAGES_PER_CLASS

    figure, axes = plt.subplots(
        rows,
        columns,
        figsize=(10, 8)
    )

    for axis, (image_path, true_class) in zip(
        axes.flatten(),
        selected_images
    ):
        predicted_class, confidence = predict_image(
            model,
            class_names,
            image_path,
            transform
        )

        with Image.open(image_path) as image:
            image = image.convert("RGB")

            axis.imshow(image)
            axis.axis("off")
            axis.set_title(
                f"True: {true_class}\n"
                f"Pred: {predicted_class}\n"
                f"Conf: {confidence:.2f}"
            )

```

```

    )

plt.tight_layout()

OUTPUT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True
)

plt.savefig(OUTPUT_PATH)
plt.close()

print(f"Saved prediction gallery: {OUTPUT_PATH}")

```

Task 7: Add Main Function

Add:

```

def main():
    class_names = load_class_names()

    model = load_model(class_names)

    create_gallery(
        model,
        class_names
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.create_transfer_gallery
```

Expected Result

The script should create:

```
reports/transfer_prediction_gallery.png
```

The figure should contain 9 images:

- 3 forest images,
- 3 residential images,
- 3 river images.

Each image should show:

- true class,
- predicted class,
- confidence.

Analysis Questions

Answer:

- Which predictions look visually correct?
- Are any wrong predictions understandable from the image?
- Does the model seem more confident on some classes than others?
- Can visual inspection reveal problems that accuracy alone does not show?

28 Visual Explanation with Grad-CAM

So far, the model has produced class predictions such as:

```
forest  
residential  
river
```

and confidence values such as:

```
0.91
```

However, this does not tell us **why** the model made a prediction.

In real Earth Observation systems, this question is important:

Which part of the satellite image influenced the model decision?

To investigate this, we can use a visual explanation method called:

```
Grad-CAM
```

Grad-CAM means:

```
Gradient-weighted Class Activation Mapping
```

What Is Grad-CAM?

Grad-CAM creates a heatmap showing which image regions were important for a selected model prediction.

The result is not a new prediction.

It is a visual explanation of an existing prediction.

A Grad-CAM heatmap may highlight:

- forest texture,
- river shape,
- building patterns,
- roads or boundaries,
- other regions that affected the model output.

The heatmap is usually overlaid on the original image.

Bright or strong regions in the heatmap mean:

The model used this region strongly when making the prediction.

Dark or weak regions mean:

This region had less influence on the prediction.

Grad-CAM helps us check whether the model behaves reasonably.

For example:

- if the model predicts **river**, the heatmap should ideally highlight the river-like structure;
- if the model predicts **forest**, the heatmap should ideally highlight vegetation texture;
- if the model predicts **residential**, the heatmap should ideally highlight regular built-up patterns.

This is useful because accuracy alone does not explain model behavior.

A model may be correct for the wrong reason.

For example, it may classify an image correctly because of:

- background color,
- image artifact,
- corner pattern,
- dataset bias,
- irrelevant visual cue.

Grad-CAM is an interpretation tool, not a formal proof that the model is correct.

It can help engineers inspect model behavior, but it should not be treated as a complete safety guarantee.

In real systems, visual explanation should be combined with:

- validation on larger datasets,
- robustness testing,
- uncertainty estimation,
- out-of-distribution detection,
- expert review.

Task 1: Create Grad-CAM Script

Create:

```
src/vision/create_gradcam.py
```

Add imports:


```

from pathlib import Path

import numpy as np
import torch
from torch import nn
from torchvision import transforms, models
from PIL import Image
import matplotlib.pyplot as plt

```

Add configuration:

```

MODEL_PATH = Path("models/resnet18_transfer.pt")
CLASS_NAMES_PATH = Path("models/resnet18_classes.txt")

IMAGE_PATH = Path("data/processed/images/test/river/river_0000.jpg")
OUTPUT_PATH = Path("reports/gradcam_example.png")

```

Task 2: Load Class Names

Add:

```

def load_class_names():
    if not CLASS_NAMES_PATH.exists():
        print(f"Error: class file not found: {CLASS_NAMES_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    with open(CLASS_NAMES_PATH, "r") as f:
        class_names = [
            line.strip()
            for line in f.readlines()
            if line.strip()
        ]

    return class_names

```

Task 3: Load the Model

Add:

```

def load_model(class_names):
    if not MODEL_PATH.exists():
        print(f"Error: model file not found: {MODEL_PATH}")
        print("Train the transfer model first.")
        raise SystemExit(1)

    model = models.resnet18(weights=None)

    input_features = model.fc.in_features

    model.fc = nn.Linear(
        input_features,
        len(class_names)
    )

    state_dict = torch.load(

```

```

        MODEL_PATH,
        map_location="cpu"
    )

    model.load_state_dict(state_dict)
    model.eval()

    return model

```

Task 4: Prepare Image

Add:

```

def load_image(image_path):
    if not image_path.exists():
        print(f"Error: image not found: {image_path}")
        raise SystemExit(1)

    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])

    display_transform = transforms.Compose([
        transforms.Resize((224, 224))
    ])

    with Image.open(image_path) as image:
        image = image.convert("RGB")
        display_image = display_transform(image)
        image_tensor = transform(image)

    image_tensor = image_tensor.unsqueeze(0)

    return image_tensor, display_image

```

Task 5: Create Grad-CAM Helper Class

Grad-CAM needs access to:

- activations from a convolutional layer,
- gradients flowing back to that layer.

For ResNet18, we will use the last convolutional block:

```
model.layer4
```

Add:

```

class GradCAM:

    def __init__(self, model, target_layer):

```

```

self.model = model
self.target_layer = target_layer

self.activations = None
self.gradients = None

self.forward_hook = target_layer.register_forward_hook(
    self.save_activations
)

self.backward_hook = target_layer.register_full_backward_hook(
    self.save_gradients
)

def save_activations(self, module, input_data, output_data):
    self.activations = output_data.detach()

def save_gradients(self, module, grad_input, grad_output):
    self.gradients = grad_output[0].detach()

def generate(self, image_tensor, target_class_index):
    self.model.zero_grad()

    outputs = self.model(image_tensor)

    score = outputs[0, target_class_index]

    score.backward()

    gradients = self.gradients[0]
    activations = self.activations[0]

    weights = gradients.mean(
        dim=(1, 2)
    )

    cam = torch.zeros(
        activations.shape[1:],
        dtype=torch.float32
    )

    for channel_index, weight in enumerate(weights):
        cam += weight * activations[channel_index]

    cam = torch.relu(cam)

    cam = cam - cam.min()

    if cam.max() > 0:
        cam = cam / cam.max()

    return cam.numpy()

def close(self):
    self.forward_hook.remove()
    self.backward_hook.remove()

```

What Happens Here?

The Grad-CAM class attaches hooks to the model.

A hook is a small function that records what happens inside the neural network.

The forward hook stores:

```
feature maps
```

from the selected layer.

The backward hook stores:

```
gradients
```

showing how strongly those feature maps influenced the selected class score.

Grad-CAM then combines these values into one heatmap.

Task 6: Predict the Image

Add:

```
def predict(model, image_tensor, class_names):
    with torch.no_grad():
        outputs = model(image_tensor)

        probabilities = torch.softmax(
            outputs,
            dim=1
        )

        confidence, predicted_index = torch.max(
            probabilities,
            dim=1
        )

        predicted_class = class_names[
            predicted_index.item()
        ]

    return predicted_index.item(), predicted_class, confidence.item()
```

Task 7: Save Grad-CAM Visualization

Add:

```
def save_gradcam_visualization(
    display_image,
    cam,
    predicted_class,
    confidence
):
    image_array = np.array(display_image)

    cam_image = Image.fromarray(
        np.uint8(cam * 255)
    )

    cam_image = cam_image.resize(
```

```

        display_image.size
    )

    cam_resized = np.array(cam_image) / 255.0

    figure, axes = plt.subplots(
        1,
        3,
        figsize=(12, 4)
    )

    axes[0].imshow(image_array)
    axes[0].set_title("Original image")
    axes[0].axis("off")

    axes[1].imshow(cam_resized, cmap="jet")
    axes[1].set_title("Grad-CAM heatmap")
    axes[1].axis("off")

    axes[2].imshow(image_array)
    axes[2].imshow(
        cam_resized,
        cmap="jet",
        alpha=0.45
    )
    axes[2].set_title(
        f"Prediction: {predicted_class}\n"
        f"Confidence: {confidence:.2f}"
    )
    axes[2].axis("off")

    plt.tight_layout()

    OUTPUT_PATH.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    plt.savefig(OUTPUT_PATH)
    plt.close()

    print(f"Saved Grad-CAM visualization: {OUTPUT_PATH}")

```

Task 8: Add Main Function

Add:

```

def main():
    class_names = load_class_names()

    model = load_model(class_names)

    image_tensor, display_image = load_image(
        IMAGE_PATH
    )

    predicted_index, predicted_class, confidence = predict(
        model,

```

```

        image_tensor,
        class_names
    )

    print("=== Grad-CAM Explanation ===")
    print(f"Image: {IMAGE_PATH}")
    print(f"Predicted class: {predicted_class}")
    print(f"Confidence: {confidence:.4f}")

    gradcam = GradCAM(
        model=model,
        target_layer=model.layer4
    )

    cam = gradcam.generate(
        image_tensor=image_tensor,
        target_class_index=predicted_index
    )

    gradcam.close()

    save_gradcam_visualization(
        display_image=display_image,
        cam=cam,
        predicted_class=predicted_class,
        confidence=confidence
    )

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.create_gradcam
```

Expected Result

The script should create:

```
reports/gradcam_example.png
```

The output image should contain:

- the original satellite image,
- the Grad-CAM heatmap,
- the heatmap overlaid on the original image.

Experiment

Change:

```
IMAGE_PATH = Path("data/processed/images/test/river/river_0000.jpg")
```

to images from other classes, for example:

```
IMAGE_PATH = Path("data/processed/images/test/forest/forest_0000.jpg")
IMAGE_PATH = Path("data/processed/images/test/residential/residential_0000.jpg")
```

Run the script again.

Observe whether the heatmap highlights meaningful regions.

Analysis Questions

Answer:

- Which regions of the image were highlighted by Grad-CAM?
- Did the highlighted regions make sense for the predicted class?
- Did the model focus on land-cover patterns or irrelevant image regions?
- Did Grad-CAM reveal anything that accuracy alone did not show?
- Would such a visualization be useful during model validation for Earth Observation systems?

29 Final Engineering Decision

You are now the engineer responsible for selecting the visual classification model for a prototype Earth Observation system.

Based on all experiments from Laboratory 8 and Laboratory 9, choose exactly one model:

- SimpleCNN trained from scratch,
- ResNet18 with frozen feature extractor,
- ResNet18 with fine-tuning.

Task

Create:

```
reports/final_engineering_decision.txt
```

Your decision must include:

- selected model,
- technical justification,
- expected advantages,
- possible risks,
- deployment considerations.

Engineering Questions

Justify your decision using:

- classification accuracy,
- training time,

- implementation complexity,
- risk of overfitting,
- robustness on unknown inputs,
- suitability for small datasets,
- expected operational reliability.

30 Submission

Submit the following artifacts:

- transfer learning training script,
- transfer learning prediction script,
- trained ResNet18 model,
- class name file,
- transfer learning report,
- transfer confusion matrix,
- fine-tuned ResNet18 model,
- fine-tuning confusion matrix,
- fine-tuning analysis,
- model comparison report,
- unknown input analysis,
- prediction gallery,
- normalization experiment results,
- larger dataset experiment results,
- Grad-CAM script,
- Grad-CAM visualization,
- Grad-CAM interpretation answers,
- final engineering decision report,
- repository link.

Your short reflection should answer:

- What is transfer learning?
- What is a pretrained model?
- What does it mean to freeze layers?
- What does it mean to replace the classifier head?

- What is fine-tuning?
- Which approach worked best: SimpleCNN, frozen ResNet18 or fine-tuned ResNet18?
- What are the engineering trade-offs between accuracy, training time and model complexity?
- Would you use transfer learning in an operational Earth Observation system? Why?

31 Final Note

This laboratory introduced transfer learning.

The key idea is:

A neural network trained on a large image dataset can be reused as a visual feature extractor for a new task.

This is especially important in space and Earth Observation systems, where labeled data may be limited and training large models from scratch may be expensive.

Transfer learning is widely used in:

- satellite image classification,
- land cover mapping,
- disaster monitoring,
- infrastructure detection,
- onboard AI prototyping,
- mission-specific visual analytics.

The most important engineering insight is:

You do not always need to train a deep model from scratch. Often, adapting a pretrained model is faster, more reliable and more practical.